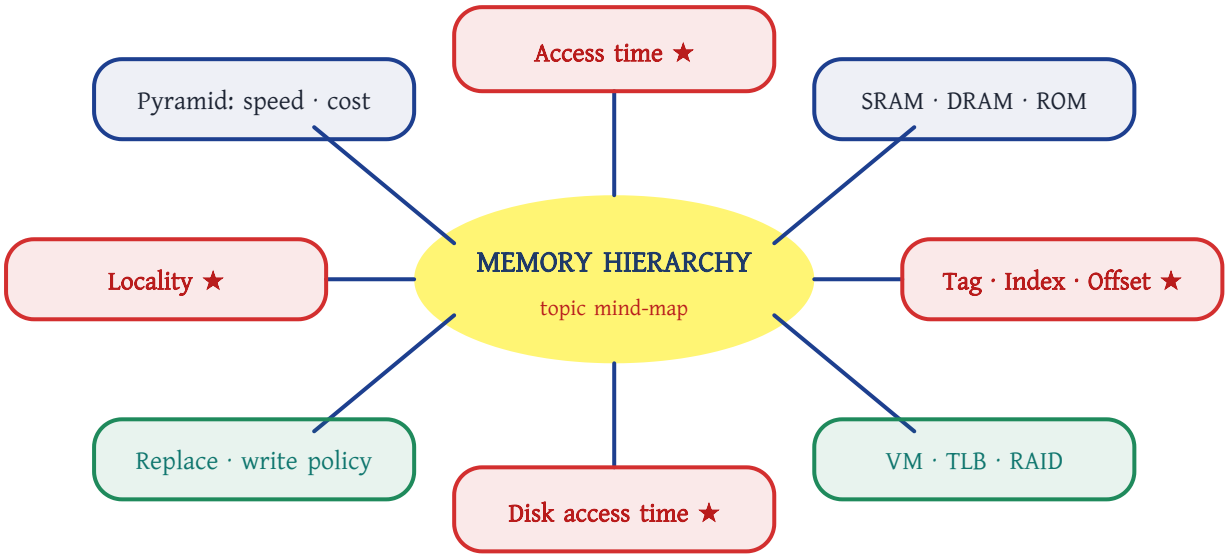
Memory Hierarchy

Locality · RAM/ROM · Cache mapping · VM/TLB · Disk: exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = policy / applied

Syllabus coverage

"Computer Architecture: … Processor design … Pipeline … **Memory organization: cache, virtual memory**, channels." Microprocessor paper: "**Memory Interfacing**". This notebook covers the hierarchy, semiconductor memory, capacity and interleaving, cache design and secondary storage; CPU, pipelining and CPI are in the CPU Organization notes, paging algorithms in the Operating Systems chapter.

High-yield list

① fastest = registers
② temporal vs spatial
③ SRAM cache, DRAM main
④ T = h·Tc + (1-h)(Tc+Tm)
⑤ tag/index/offset bits
⑥ write-back + dirty bit
⑦ seek + latency + transfer

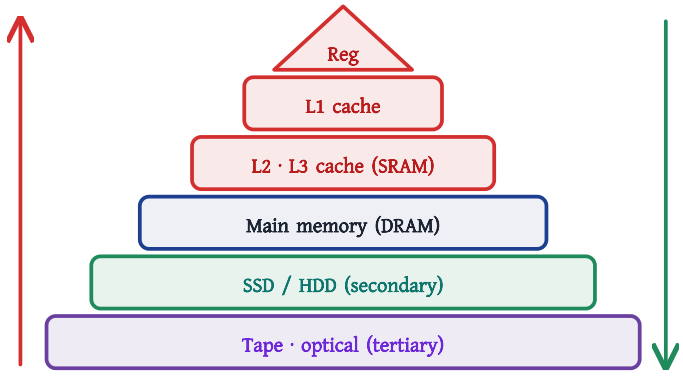
Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Hierarchy pyramid & locality ..... p. 2
✓ SRAM, DRAM & ROM types ..... p. 3
✓ Capacity, chips, interleaving ..... p. 4
✓ Cache basics & access time ..... p. 5
✓ Mapping: tag / index / offset … p. 6
✓ Mapping examples, replacement … p. 7
✓ Write policy, 3 Cs, multilevel … p. 8
✓ Virtual memory, TLB, disk, RAID … p. 9
✓ Night-before sheet ..... p. 10
✓ Practice MCQs + written ..... p. 11

1 Why a Memory Hierarchy?

Definition: Memory built as **levels**, each smaller, faster and **costlier per bit** than the one below. Thanks to locality the system runs at **nearly the speed of the fastest level at nearly the cost of the cheapest**.



primary = what the CPU addresses directly (top 4)

red ↑ faster, costlier/bit · green ↓ bigger, cheaper

Going DOWN the pyramid:

- capacity ↑
- access time ↑ (slower)
- cost per bit ↓
- frequency of access by the CPU ↓

Properties: **inclusion** (level i ⊂ level i+1), **coherence** (copies agree), **locality**. 90/10 rule: ≈ **90%** of time is spent in 10% of the code.

Level	Typical size	Access time	Built from	Managed by
Registers	< 1 KB	**< 1 ns (1 cycle)**	flip-flops	compiler
L1 / L2 / L3	32 K / 1 M / 32 MB	1 / 4 / 15 ns	SRAM	hardware
Main memory	8 – 64 GB	50 – 100 ns	DRAM	OS
SSD / HDD	0.5 – 20 TB	0.1 ms / 5 – 10 ms	flash / disk	OS

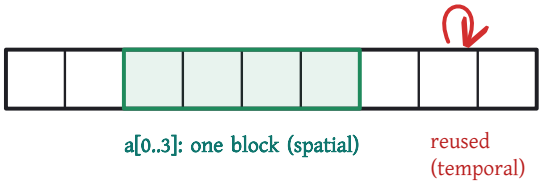
Exam trap: Fastest = **registers**, not cache. RAM ≈ **150 – 300 cycles** at 3 GHz.

2 Locality of Reference

Temporal: an item used now is **used again soon** (loop counter, *sum*, loop code). Exploited by **keeping** recent items (LRU).

Spatial: **nearby addresses** are used soon (arrays, sequential code). Exploited by fetching a whole **block** (64 B) and prefetching.

```
sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];    /* a[i]: spatial */
                    /* sum, i: temporal */
```

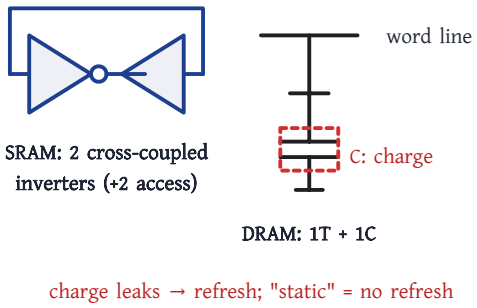


one miss brings the neighbours too

Memory trick: Temporal = same Thing, again in Time. Spatial = Surrounding Space. C is row-major: loop *a[i][j]* with **j inside**.

3 RAM: SRAM vs DRAM

	SRAM	DRAM
Cell	**6T** flip-flop	**1T + 1C**
Refresh	none	every ≈ 64 ms
Speed	faster	slower
Density	low	high
Cost / bit	high	low
Read	non-destructive	destructive
Used as	cache	main memory



charge leaks → refresh; "static" = no refresh

> **Exam trap:** "Static" does **not** mean non-volatile: SRAM and DRAM **both lose data** when power goes. DRAM's row and column addresses share pins (multiplexed with **RAS / CAS**), so a 1M × 1 DRAM needs only 10 address pins, not 20.

DRAM family: asynchronous DRAM → FPM → EDO → **SDRAM** (clocked) → **DDR** (data on **both clock edges**) → DDR2 / 3 / 4 / 5. Peak rate = transfers/s × bus width: DDR4-3200 on a 64-bit (8 B) channel = 3200 × 10^6 × 8 B = **25.6 GB/s**.

SRAM → cache, DRAM →

4 ROM Family (Non-volatile)

Type	Written	Erased	Typical use
Mask ROM	by the maker (mask)	never	mass-produced code
PROM (OTP)	once by user (fuses)	never	small-run firmware
EPROM	user, in a programmer	**UV light**, whole chip	prototypes (window)
EEPROM	electrically, in circuit	electrically, **per byte**	settings, serial numbers
Flash	electrically, in circuit	electrically, **per block**	UEFI/BIOS, SSD, USB

NOR flash: random byte read, execute-in-place → firmware. **NAND flash**: denser, page read / block erase → SSDs, cards; limited erase cycles (wear levelling).

> **Memory trick:** EPROM = **E**rase with **P**hotons (UV). EEPROM: extra E = **E**lectrically. Flash erases a whole block "in a flash".

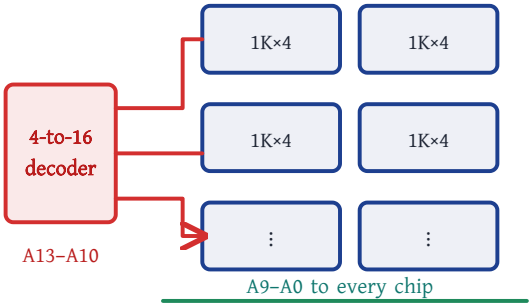
> **Exam trap:** ROM is also **random access** ("RAM" really means read/write memory), and ROM is primary but non-volatile. Access methods: **random** (RAM, ROM) · **sequential** (tape) · **direct** (disk) · **associative**, by content (cache tags, TLB).

5 Memory Organisation & Capacity Maths

Key formulas: 2^n **words × m bits → n address lines, m data lines** · chips = (words ÷ chip words) × (width ÷ chip width) · address bits = $\log_2$(**addressable units**)

Width (word-size) expansion: chips side by side share **all** address lines; each supplies some data bits (two 1K × 4 → 1K × 8).

Depth (word-count) expansion: extra high-order address bits go to a **decoder** whose outputs drive the chips' **chip select (CS)**.



16K × 8 from 1K × 4: 16 rows × 2 chips

Worked examples ✍️:

1. A 4K × 8 chip: 4K = 2^{12} → **12** address lines, **8** data lines; capacity 32 Kbit = 4 KB
2. 16K × 8 memory from 1K × 4 chips: (16K / 1K) × (8 / 4) = 16 × 2 = **32 chips**; 14 address lines: A9–A0 to every chip, A13–A10 into a **4-to-16 decoder**
3. 64 MB, byte-addressable: 2^{26} bytes → **26** address bits. Same memory, 32-bit (4-byte) words, word-addressable: 2^{24} words → **24** bits
4. 32-bit byte address → 2^{32} B = **4 GB**; 8086 (20 lines) → 1 MB

Exam trap: Check **byte**- vs **word**-addressable before taking $\log_2$.

6 Memory Interleaving

- **k modules (banks)** work in parallel
- **Low-order**: low address bits pick the module (address mod k); consecutive words in **different** modules → a block is fetched in parallel
- **High-order**: high bits pick the module; consecutive words in the **same** module → easy expansion, no speed-up



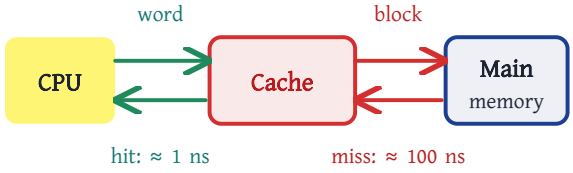
4-way low-order interleaving

Worked example ✍️: 4-word block; 1 cycle to send the address, 15 cycles per DRAM access, 1 cycle per word on the bus. One-word-wide memory: 1 + 4 × 15 + 4 × 1 = **65 cycles**. 4-bank interleaved: 1 + 15 + 4 × 1 = **20 cycles** (3.25× faster miss penalty).

7 Cache Memory: the Basics

Definition: A small, fast **SRAM** memory between the CPU and main memory that holds copies of recently used **blocks** (lines). Invisible to the programmer: managed entirely by hardware.

- **Block / line**: the unit moved between cache and memory (typ. 64 B)
- **Hit**: word found in cache · **Miss**: not found → fetch the block
- **Hit ratio h = hits / (hits + misses)**; miss ratio = 1 − h
- **Hit time** (Tc) · **miss penalty** (time to bring the block)
- Each line stores a **tag** (which block is here), a **valid bit**, and a **dirty bit** for write-back



CPU ↔ cache: words · cache ↔ RAM: blocks

8 Average Access Time: Which Formula?

Organisation	A miss costs	Formula
Hierarchical (look-through)	Tc + Tm	**T = h·Tc + (1 − h)(Tc + Tm)**
Simultaneous (parallel)	Tm only	T = h·Tc + (1 − h)·Tm
AMAT (Hennessy–Patterson)	miss penalty	AMAT = hit time + miss rate × miss penalty

Hierarchical: the cache is checked first, then memory. Simultaneous: both are started together. Hierarchical simplifies to Tc + (1 − h)·Tm, the AMAT form with penalty = Tm. **Default** when the question is silent: **hierarchical**, but check whether the options hold the simultaneous answer.

Worked examples ✍️:

1. Tc = 10 ns, Tm = 100 ns, h = 0.9: hierarchical 0.9 × 10 + 0.1 × 110 = **20 ns**; simultaneous 9 + 10 = **19 ns**; speed-up = 100 / 20 = **5**
2. Classic: Tc = 100 ns, Tm = 1000 ns, h = 0.9 → hierarchical **200 ns** (simultaneous 190 ns)
3. AMAT: hit 1 ns, miss rate 5%, penalty 100 ns → 1 + 0.05 × 100 = **6 ns**
4. Find h: Tc = 5, Tm = 80, target 9 ns: 9 = 5 + (1 − h) × 80 → **h = 0.95**
5. CPI (see CPU Organization): base 1, 1.3 accesses / instr., miss 2%, penalty 100 → 1 + 1.3 × 0.02 × 100 = **3.6**

Memory trick: "**Through**" the cache: a miss **pays both**, Tc + Tm. "**Side by side**": a miss pays Tm only.

Exam trap: Given the **miss** ratio (0.1)? Don't use it as h. Check: Tc ≤ T ≤ Tc + Tm.

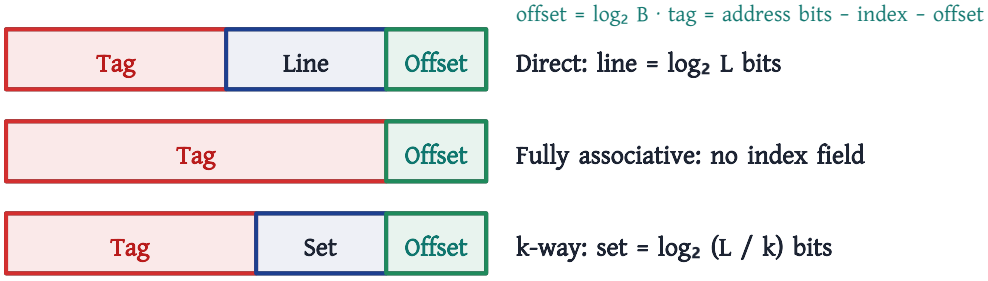
9 Cache Mapping Techniques

Mapping decides **where** a memory block may be placed in the cache, and so how the address is split. Let block size = B bytes, lines = L, associativity = k, sets S = L / k.

Direct: block j → line **j mod L**. One place only, **one comparator**. Simple, fast; suffers **conflict misses** (thrashing).

Fully associative: block → **any** line; all tags compared **in parallel** (L comparators, CAM). No conflict misses; costly.

k-way set assoc.: block j → set **j mod S**, any of its k lines; k comparators. The compromise used in real CPUs.



the three address splits (higher associativity → longer tag, shorter index)

	Direct	k-way set associative	Fully associative
Places for a block	1	k	L (any)
Tag comparators	1	k	L
Conflict misses	most	fewer	**none**
Hardware cost, hit time	lowest	medium	highest
Replacement policy needed?	**no** (no choice)	yes, within the set	yes
Special case of k-way	k = 1	1 < k < L	k = L (one set)

Same cache, different associativity

64 KB · 64 B · 32-bit	Tag	Index	Offset
Direct (1024 lines)	16	10	6
2-way (512 sets)	17	9	6
4-way (256 sets)	18	8	6
8-way (128 sets)	19	7	6
Fully associative	26	–	6

Memory trick: **Doubling k** moves one bit from index to tag. Direct: **tag = log₂(MM ÷ cache)** = 32 − 16 = 16.

Exam trap: Offset ← **block** size; index ← **sets** (not lines) when k > 1.

Where does block 1000 go? ✍️: Direct, 128 lines: 1000 mod 128 = **line 104**. 4-way, 32 sets: 1000 mod 32 = **set 8** (any of its 4 lines).

10 Mapping: Worked Examples

Example 1: split an address ✍️: Direct-mapped 64 KB cache, 64 B blocks, 32-bit address 0x12345678 (tag 16 | line 10 | offset 6): offset = 0x78 AND 0x3F = **56** · line = (0x12345678 ÷ 64) mod 1024 = **345** · tag = upper 16 bits = **0x1234**

Example 2: 2-way set associative ✍️: 32 KB cache, 32 B blocks, 32-bit address. Offset = $\log_2 32$ = **5**; lines = 32 KB / 32 B = 1024; sets = 1024 / 2 = 512 → set = **9**; tag = 32 − 9 − 5 = **18**.

Example 3: tag memory ✍️: Example 1's cache: 1024 lines × (16 tag + 1 valid) = 17,408 bits = **2176 B** on top of the 64 KB of data.

Example 4: block trace 0, 8, 0, 6, 8 in a 4-block cache

Cache	0	8	0	6	8	Misses
Direct (j mod 4)	M	M	M	M	M	5
2-way, LRU	M	M	**H**	M	M	4
Fully assoc.	M	M	**H**	M	**H**	**3**

Direct: 0 and 8 share line 0 and evict each other. 2-way: 0, 6 and 8 all fall in set 0 (3 blocks, 2 ways). Fully assoc.: only the 3 cold misses. More ways remove **conflict misses**.

11 Replacement Policies

Needed only when the block has a **choice** of lines (set or fully associative) and all are full.

Policy	Evicts the block ...	Notes
LRU	least recently **used**	temporal locality; 2-way: 1 bit per set
FIFO	loaded **earliest**	round-robin pointer; **Belady's anomaly**
Random	picked at random	no bookkeeping; near LRU when large
LFU	least frequently used	needs counters
Optimal (Belady)	used farthest in future	needs the future: benchmark only

Worked trace ✍️: Fully associative, 3 lines (victim as subscript)

Ref	7	0	1	2	0	3	0	4	2	3	Miss
FIFO	M	M	M	M_7	H	M_0	M_1	M_2	M_3	M_0	9
LRU	M	M	M	M_7	H	M_1	H	M_2	M_3	M_0	**8**
OPT	M	M	M	M_7	H	M_1	H	M_0	H	H	**6**

Memory trick: FIFO: **birth date** · LRU: **last visit** · OPT: **next visit** (same algorithms in paging).

12 Write Policies

On a write hit	Write-through	Write-back (copy-back)
Memory updated	on **every** write	when a **dirty** block is evicted
Memory traffic	high (write buffer hides it)	low
Memory up to date?	always (safe for DMA)	no: dirty bit, coherence care
Usual write-miss partner	**no-write-allocate**	**write-allocate**

Write-allocate (fetch-on-write): on a write miss, load the block, then write it. **No-write-allocate** (write-around): write straight to memory.

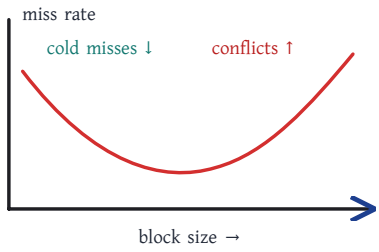
Traffic ✍: A loop writes one variable 100 times: write-through **100** memory writes, write-back **1** (at eviction).

Write-back time ✍: Tc = 5 ns, Tm = 50 ns, h = 0.9, 30% of victims dirty: T = 5 + 0.1 × (50 + 0.3 × 50) = **11.5 ns**

Multicore: several caches may hold copies; a **coherence protocol** (MESI, snooping) keeps them consistent.

13 Why Misses Happen: the 3 Cs

Miss	Cause	Remedy
Compulsory	first access (cold)	bigger block, prefetch
Capacity	working set > cache	bigger cache
Conflict	blocks clash in a set	more ways, victim cache
Coherence	another core wrote it	(the 4th C)



block size: U-shaped trade-off

Memory trick: **Cold** start, **Crowded** cache, **Clash** in a set. Fully associative: **no** conflict misses.

Exam trap: A bigger block cuts cold misses but raises the **miss penalty**.

14 Multilevel Caches

- **L1**: split I + D caches, per core; aim: **short hit time**
- **L2**: unified, per core; aim: low miss rate
- **L3**: shared by all cores (last level)
- **Local** miss rate = misses / accesses to **that level**; **global** = misses / all CPU accesses

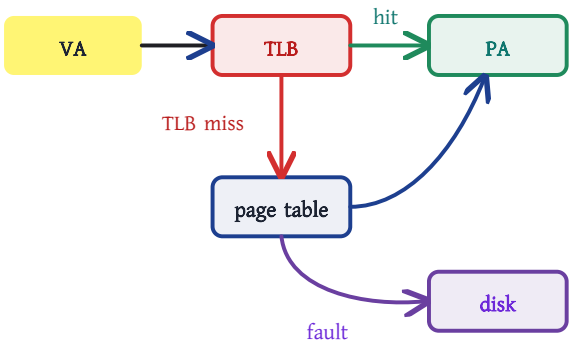
Two-level AMAT ✍️: L1: 1 ns, miss 5%. L2: 10 ns, local miss 20%. RAM 100 ns.
AMAT = 1 + 0.05 × (10 + 0.2 × 100) = 1 + 0.05 × 30 = **2.5 ns**
Without L2: 1 + 0.05 × 100 = 6 ns. Global L2 miss rate = 0.05 × 0.2 = **1%**.

15 Virtual Memory & TLB (Summary)

VM uses main memory as a **cache for the disk**. The **MMU** translates virtual addresses page by page through the **page table** (in RAM, found via the PTBR).

A page fault costs **millions of cycles** → large pages (4 KB), fully associative placement, LRU-like replacement, always **write-back**.

TLB = small associative cache of recent page-table entries.



Cache	VM	Formula / example
block	page / frame	pages = 2^(VA bits − offset bits)
miss	page fault (OS)	32-bit VA, 4 KB page → 2^{20} PTEs × 4 B = **4 MB**
tag store	page table, TLB	64 MB RAM, 4 KB pages → 2^{14} frames

EAT with a TLB ✍️: TLB 20 ns, memory 100 ns, TLB hit 0.8, one-level table: 0.8 × (20 + 100) + 0.2 × (20 + 100 + 100) = **140 ns** (a TLB miss adds one memory access). Paging algorithms, thrashing → OS chapter.

16 Secondary Storage: Disk & RAID

Disk formulas: **Access time = seek + rotational latency + transfer** (+ controller overhead) · average latency = ½ × (60 / rpm) s · capacity = surfaces × tracks × sectors/track × bytes/sector

Worked example ✍️: 6000 rpm, average seek 8 ms, 100 sectors × 512 B per track.

1. Rotation = 60 / 6000 s = 10 ms → latency **5 ms**; 1 sector = 10 / 100 = 0.1 ms
2. Access = 8 + 5 + 0.1 = **13.1 ms**; 4 KB (8 sectors) = 13.8 ms; rate = 51,200 B / 10 ms = **5.12 MB/s**
3. 16 surfaces × 1024 tracks × 128 sectors × 512 B = 2^{30} B = **1 GB**

RAID	Scheme	Min	Usable	Survives
0	striping, no redundancy	2	n	no failure
1	mirroring	2	n / 2	1 disk per pair
5	striping + **distributed parity**	3	n - 1	1 disk
6	two distributed parities	4	n - 2	2 disks
10	mirror, then stripe	4	n / 2	1 per pair

RAID 2 = bit-level Hamming, 3 = byte-level, 4 = block-level, each with a **dedicated** parity disk (bottleneck). Parity = XOR. 5 × 2 TB: RAID 5 → **8 TB**, RAID 6 → 6 TB.

Night-Before-the-Exam Sheet

Fact sheet:

Reg → L1 → L2 → L3 → RAM → disk → tape

Down: size ↑, time ↑, cost/bit ↓

Temporal = same item · spatial = neighbours

SRAM 6T, cache · DRAM 1T1C, refresh

EPROM: UV · EEPROM: byte · flash: block

$2^n \times m \rightarrow$ n address, m data lines

Chips = (W/w) × (M/m)

Low-order interleave: module = addr mod k

Hier.: T = h·Tc + (1-h)(Tc+Tm)

Simult.: T = h·Tc + (1-h)·Tm

AMAT = hit time + miss rate × penalty

offset = $\log_2$ B · sets = L / k

tag = address - index - offset

Direct tag = $\log_2$(MM / cache)

EAT(TLB) = h(t+m) + (1-h)(t+2m)

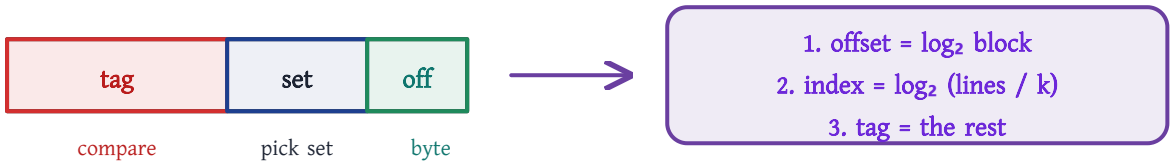
Disk = seek + ½ rotation + transfer

Top 12 one-liners for MCQs

1. Fastest storage: CPU registers
2. Cache sits between CPU and main memory
3. Cache is made of SRAM
4. DRAM needs refreshing; SRAM does not
5. Loop variable reuse = temporal locality
6. Any block → any line = fully associative
7. Direct mapping needs no replacement policy
8. Dirty bit belongs to write-back
9. FIFO can show Belady's anomaly
10. TLB = cache of page-table entries
11. RAID 5 = distributed parity, n - 1
12. 7200 rpm → 4.17 ms average latency

Top traps: registers, not cache, are fastest · SRAM is volatile too · hierarchical (Tc + Tm on a miss) vs simultaneous (Tm) · index counts **sets** when k > 1 · byte vs word addressing · latency is **half** a rotation · RAID 0 has no redundancy · bigger blocks are not always better

All tricks together: Temporal = same Thing in Time, Spatial = Surrounding Space · EPROM = Erase with Photons · "through" pays Tc + Tm, "side by side" pays Tm · doubling k moves a bit from index to tag · FIFO birth date, LRU last visit, OPT next visit · 3 Cs: Cold, Crowded, Clash



the 3-step address split: always offset first, tag last

✓ Practice MCQs (exam pattern, 10 marks)

1. Which memory needs periodic refresh?
(a) SRAM (b) DRAM
(c) EPROM (d) Flash

2. Hierarchical: Tc 20 ns, Tm 200 ns, h 0.9. T = ?
(a) 38 ns (b) 36 ns
(c) 40 ns (d) 22 ns

3. 16K × 8 from 1K × 4 chips needs:
(a) 32 chips (b) 16 chips
(c) 8 chips (d) 64 chips

4. Direct, 32 KB, 16 B blocks, 32-bit: tag bits?
(a) 15 (b) 21
(c) 11 (d) 17

5. The dirty bit is needed in:
(a) write-back (b) write-through
(c) direct mapping (d) FIFO

6. Two blocks fighting for one line cause a:
(a) compulsory miss (b) capacity miss
(c) conflict miss (d) coherence miss

7. Scanning an array in order exploits:
(a) temporal locality (b) spatial locality
(c) interleaving (d) write-allocate

8. 7200 rpm disk, average latency:
(a) 8.33 ms (b) 7.2 ms
(c) 2.08 ms (d) 4.17 ms

9. Striping with distributed parity is:
(a) RAID 5 (b) RAID 1
(c) RAID 4 (d) RAID 0

10. k-way cache with k = number of lines is:
(a) direct mapped (b) 2-way
(c) fully associative (d) interleaved

Answer key → 1-b · 2-c · 3-a · 4-d · 5-a · 6-c · 7-b · 8-d · 9-a · 10-c

Written question: Explain the three cache mappings. For a 256 KB, 4-way cache with 64 B blocks and 32-bit addresses, find the field widths, the set and tag of 0x0001F2C4, and the tag storage (1 valid bit per line). 8 marks

1. Direct: line = j mod L · fully associative: any line · k-way: set = j mod S (one merit / demerit each) 3
2. Offset = $\log_2 64$ = **6**; 4096 lines ÷ 4 = 1024 sets → **10**; tag = 32 − 10 − 6 = **16** 2
3. Offset = 0xC4 AND 0x3F = **4**; set = ⌊0x1F2C4 ÷ 64⌋ mod 1024 = 1995 mod 1024 = **971**; tag = **0x0001** 2

4 Tag storage = 4096 × (16 + 1) = 69,632 bits = **8.5 KB** 1

Before the exam ✓

- ✓ Know which access-time formula to use
- ✓ Can split any address into tag / index / offset
- ✓ Can trace FIFO and LRU on a block string
- ✓ Can compute disk access time and RAID capacity

Also practise (written):

① SRAM vs DRAM, 6 points
② 32K × 16 from 4K × 8 chips
③ write-through vs write-back
④ two-level AMAT with global miss rate